Support Vector Machines

DSC 340 Applied Machine Learning

March 17

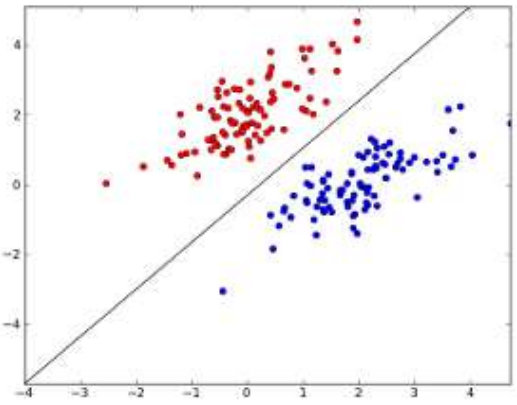
Linear classification models revisited

- **Logistic regression**

$$\mathrm{P}(1|\mathbf{x}) = \frac{1}{(1+e^{-z})}, \qquad z = \mathbf{w}\cdot\mathbf{x} + \mathrm{w}_0$$

Note: Linear regression minimizes the **mean squared error** (MSE). Logistic regression finds a boundary that maximizes the **probability (likelihood)** of correctly classifying observed data. Equivalently, it minimizes the **log-loss** (also called **cross-entropy**).

- Soon: Perceptron
- Decision boundary: **hyperplanes**
  - 1 feature: point
  - 2 features: line
  - 3 features: plane
  - m features: $m-1$ dim hyperplane



Linear regression minimizes the **mean squared error** (MSE). Logistic regression finds a boundary that maximizes the **probability (likelihood)** of correctly classifying observed data. Equivalently, it minimizes the **log-loss** (also called **cross-entropy**).

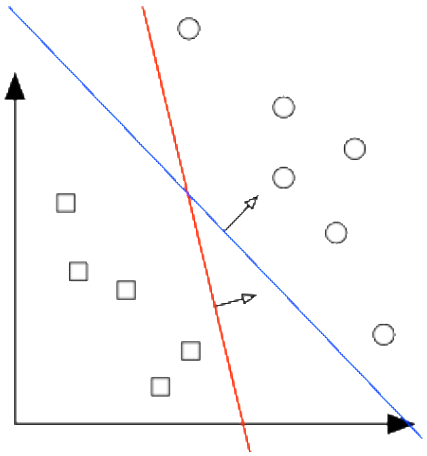
Not linear models

- KNN classifiers
- Decision trees
  - Splits often linear, but recursively applied
- Random forests
- Soon: Naïve Bayes classifiers (NLP/text classification)

SVM Big Idea

- Transform features
  - New coordinate system with more dimensions
  - For now, think of adding columns that are functions of existing columns
- Maximum margin classifier

Maximum margin classifier

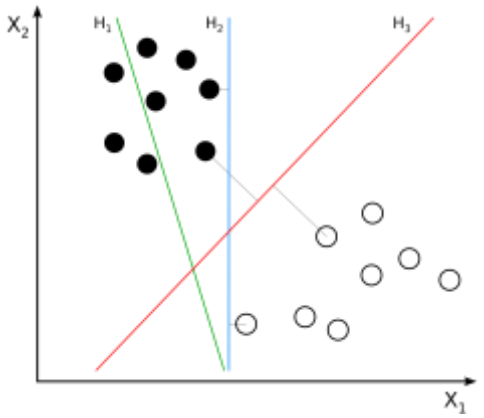


Maximum margin classifier

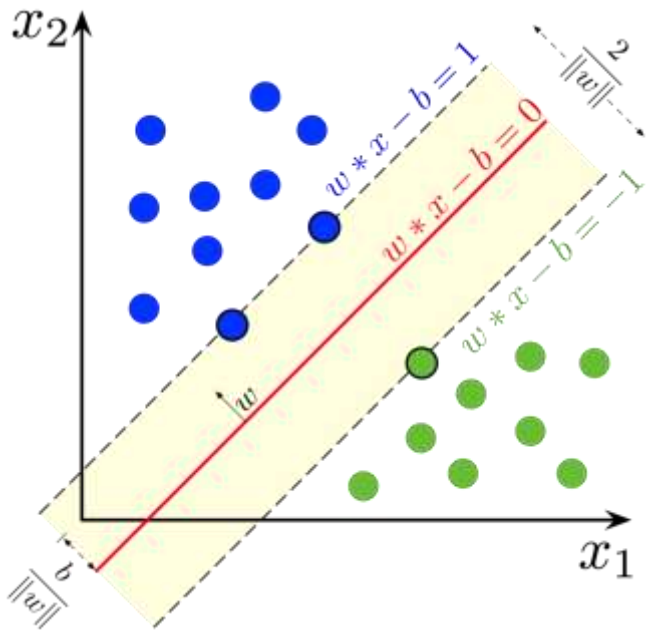
- **Margin** …

  *Distance of **point** …*

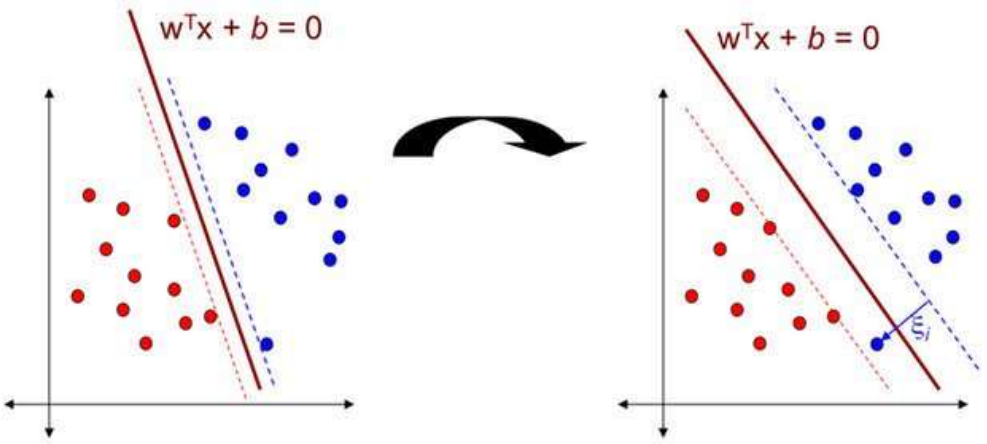
  to decision boundary
- Maximum margin classifier
- Reasonable choice of h-plane
- Note: In general, different from:
  - logistic regression
  - perceptron learning algorithm



Support vector classifier



Soft margin



Soft margin

- Reason: **maximum margin classifiers** susceptible to **outliers**
- **Soft margin** allows **misclassifications** (intentionally!)
- Necessary if data not **linearly separable**
- **Bias-variance tradeoff**
- Can explicitly **balance** these for a particular problem: **hyperparameter**

History

- Vladimir Vapnik
- Alexey Chervonenkis
- Isabelle Guyon:







- Idea: use a **kernel trick**!



What is the "Kernel Trick" in SVMs?

Imagine you're trying to classify two groups of points that can't easily be separated by a simple straight line (or a flat plane). A **kernel** is a clever mathematical trick that transforms your data into a higher-dimensional space, making it easier for the SVM to find a clear boundary to separate the two groups.

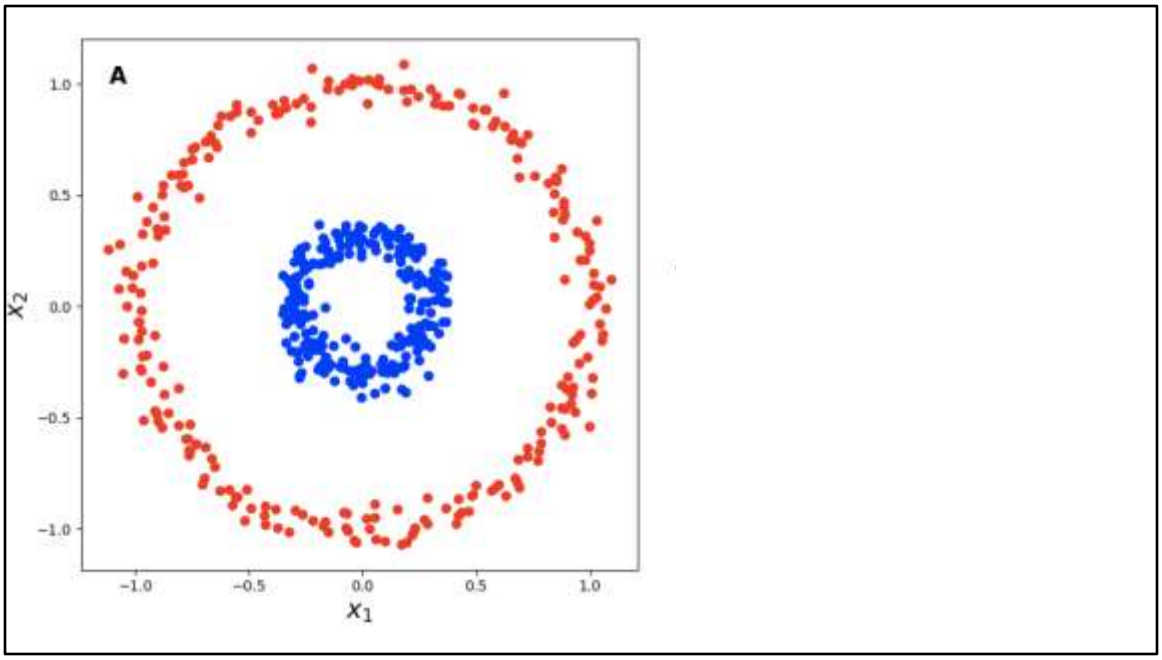
Analogy: Imagine you have salt and pepper mixed on a table, and it's hard to separate them by drawing a straight line. But if you could lift the pepper grains slightly above the salt grains into a new dimension (like levitating them upward), now they're easy to separate using a flat surface placed between them. Kernels do something similar with data points.

In short, kernels help SVMs handle complex datasets by changing the way the data is represented, making classification tasks easier.

You don’t have to explicitly calculate complicated mappings, either. That's what makes kernels powerful: they simplify complex transformations behind the scenes.

Bottom line: A kernel is a way of helping an SVM "see" data clearly by reshaping it mathematically, making previously tricky separations straightforward.


CENTRE
2


A
1.0
0.5
0.0
−0.5
−1.0
X_2
−1.0
−0.5
0.0
0.5
1.0
X_1

Kernel trick

- **Lift data** (project) into high (or infinite!) dimensional space
- Contrast: project onto **subspace**
- **Basis functions**
- Lifting: computational expensive
- Certain functions let us work in higher dimensional space …
- without actually computing …
- the transformed feature values in the space.
- Uses a **dot product**: **kernel trick**.

Mathematical Basis

- **Dot product** of vectors $\mathbf{a} \cdot \mathbf{b}$

$$\mathbf{a} \cdot \mathbf{b} = a_1 b_1 + a_2 b_2 + \cdots + a_M b_M = \sum_{j=1}^{M} a_j b_j = \mathbf{a}^{\mathrm{T}} \mathbf{b}$$

- Important to all linear models
- **Kernel function** k can be any function that satisfies the property:

$$k(\mathbf{x}, \mathbf{x}') = \varphi(\mathbf{x}) \cdot \varphi(\mathbf{x}')$$

- *Why is this useful?*
- *Kernels give a way to compute dot products in some feature space …*
- *without even knowing what this space is and what is* $\varphi(\boldsymbol{x})$.

https://stats.stackexchange.com/questions/152897/how-to-intuitively-explain-what-a-kernel-is

Why kernel tricks are useful

- By lifting data into high dimensional space …
- Better linear separability
- Computationally efficient due to kernel trick
- Doesn't require manually trying a bazillion basis functions
- Does require selecting the type of kernel
- More complex model: still parametric, but a LOT of parameters

Famous kernels you should know

- linear
- polynomial
- radial basis function: RBF
- there are many more!

$k(\mathbf{x}, \mathbf{x}') = k(\mathbf{x}, \mathbf{x}')$ Not on test. ☺

$k(\mathbf{x}, \mathbf{x}') = (\mathbf{x} \cdot \mathbf{x}')^d$

$k(\mathbf{x}, \mathbf{x}') = \exp(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2), \gamma > 0$

- "How to Select SVM kernels" (links on Moodle)
  - https://www.kdnuggets.com/2016/06/select-support-vector-machine-kernels.html

Q.: Are SVMs linear models?

- A.: Yes and no.
- If linear kernel, yes.
- If we mean, can they only classify linearly separable data, no.
- Can find decision boundaries in the original space that are not linear.
- But linear in the transformed feature space.

Parameters and hyperparameters

- Parameters: support vectors (in the transformed space)
- Hyperparameters:
- Choice of kernel
- Regularization constant C representing bias-variance tradeoff
- There are many more …
- See resources posted to Moodle

Assessment

Advantages

- Can learn non-linear boundaries
- Wide selection of kernels
- Robust against overfitting
- Even in high-dimensional space
- Even when columns > rows
- Preferred ML method in 1990s

Disadvantages

- Memory intensive
- Finding right kernel can be tricky
- Don't scale well to large data sets
- Other methods preferred today
  - Random forests, neural nets